I2C Driver Development

for the STM32L452RE (Bare-Metal Programming)

Based on the STM32L452RE Reference Manual (RM0394)
and its I2C peripheral

Table of Contents

Note: This document explains why each register is configured, not just how, and is written specifically for the STM32L452RE's new-generation I2C peripheral.

Lecture 1 — Driver API Requirements & Initial Setup

Lecture Objective

In this lecture, we begin developing a bare-metal I2C driver for the STM32L452RE microcontroller. The objectives are to:

- Create the driver source and header files.
- Understand the STM32L4 I2C peripheral architecture.
- Modify the MCU-specific header file.
- Design the APIs required by the application.
- Identify all user-configurable parameters.

1. STM32L452RE I2C Peripheral Overview

The STM32L452RE contains the new-generation STM32 I2C peripheral. Major improvements include:

- TIMINGR replaces CCR and TRISE
- ISR replaces SR1/SR2
- ICR replaces manual flag clearing
- Improved interrupt support
- Automatic END mode
- Reload mode
- SMBus support
- Better digital filtering

The STM32L452RE contains three I2C peripherals.

Peripheral	Bus
I2C1	APB1
I2C2	APB1
I2C3	APB1

All peripherals receive their clock from the APB1 bus.

2. Driver Files

Inside the Drivers directory, create:

```
Drivers
|
├── stm321452xx_i2c_driver.c
└── stm321452xx_i2c_driver.h
```

stm32l452xx_i2c_driver.c

Contains:

- Driver implementation
- Initialization
- Master transmit
- Master receive
- Slave APIs
- Interrupt handling

stm32l452xx_i2c_driver.h

Contains:

- API declarations
- Register bit macros
- Configuration structures
- Handle structures
- User macros

3. Update the MCU Header

Update `stm321452xx.h` to include:

- I2C base addresses
- Register definition structure
- Peripheral macros

- RCC enable macros
- RCC reset macros
- Bit position macros

4. I2C Register Structure

The register map is different from older STM32 families:

```
typedef struct
{
    volatile uint32_t CR1;
    volatile uint32_t CR2;
    volatile uint32_t OAR1;
    volatile uint32_t OAR2;
    volatile uint32_t TIMINGR;
    volatile uint32_t TIMEOUTR;
    volatile uint32_t ISR;
    volatile uint32_t ICR;
    volatile uint32_t PECR;
    volatile uint32_t RXDR;
    volatile uint32_t TXDR;
}I2C_RegDef_t;
```

Notice that:

- CCR no longer exists.
- TRISE no longer exists.
- SR1 and SR2 have been replaced by ISR.
- DR has been divided into RXDR and TXDR.

5. Peripheral Base Addresses

```
#define I2C1_BASEADDR
#define I2C2_BASEADDR
#define I2C3_BASEADDR
```

Peripheral pointers:

```
#define I2C1
#define I2C2
#define I2C3
```

6. Clock Enable Macros

The driver must provide macros such as:

```
I2C1_PCLK_EN()
I2C2_PCLK_EN()
I2C3_PCLK_EN()
```

Similarly:

```
I2C1_PCLK_DI()
I2C2_PCLK_DI()
I2C3_PCLK_DI()
```

These macros manipulate the RCC APB1 enable register.

7. Driver APIs

The driver should expose the following APIs.

Initialization — I2C_Init()

Responsibilities:

- Configure TIMINGR
- Configure own address
- Configure filters
- Configure ACK
- Configure addressing mode
- Enable peripheral

Master Transmission — I2C_MasterSendData()

Responsibilities:

- Generate START

- Send address
- Transmit bytes
- Generate STOP

Master Reception — I2C_MasterReceiveData()

Responsibilities:

- START
- Address phase
- Read RXDR
- STOP

Slave Transmission — I2C_SlaveSendData()

Slave waits until addressed.

Slave Reception — I2C_SlaveReceiveData()

Slave responds only after the master initiates communication.

Interrupt APIs

```
I2C_IRQInterruptConfig()
I2C_IRQPriorityConfig()
I2C_IRQHandling()
```

Generic APIs

```
I2C_PeriClockControl()
I2C_PeripheralControl()
I2C_DeInit()
I2C_GetFlagStatus()
```

8. User Configuration Structure

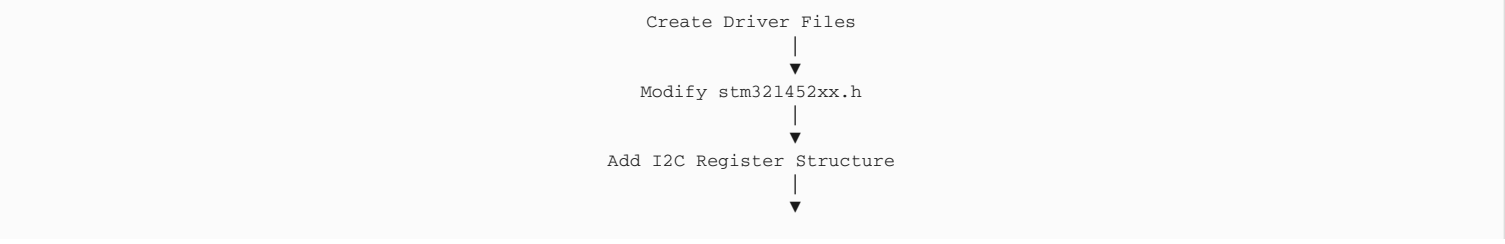
```
typedef struct
{
    uint32_t I2C_Timing;

    uint16_t I2C_OwnAddress;

    uint8_t I2C_AddressingMode;

    uint8_t I2C_ACKControl;

    uint8_t I2C_AnalogFilter;

    uint8_t I2C_DigitalFilter;

}I2C_Config_t;
```

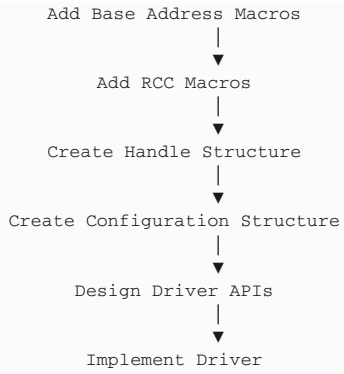
SCL speed is no longer configured directly. Instead, the user provides a TIMINGR configuration value.

9. User Configurable Parameters

Parameter	Purpose
TIMINGR	Configures I2C timing
Own Address	Slave address
Address Mode	7-bit or 10-bit
ACK Control	Enable/Disable acknowledgement
Analog Filter	Noise suppression
Digital Filter	Programmable digital filtering

10. Driver Development Flow





Key Takeaways

This version is already adapted for the STM32L452RE architecture rather than being a simple rename. In the next lecture, we build the `I2C_Handle_t` and `I2C_Config_t` structures in more detail and implement them in the driver exactly as required for the STM32L452RE.

Lecture 2 — I2C Handle Structure & Configuration Structure

Lecture Objective

In this lecture, we design the two most important data structures required by the STM32L452RE I2C driver:

- **I2C_Config_t** – Stores all user-configurable settings.
- **I2C_Handle_t** – Combines the selected I2C peripheral with its configuration.

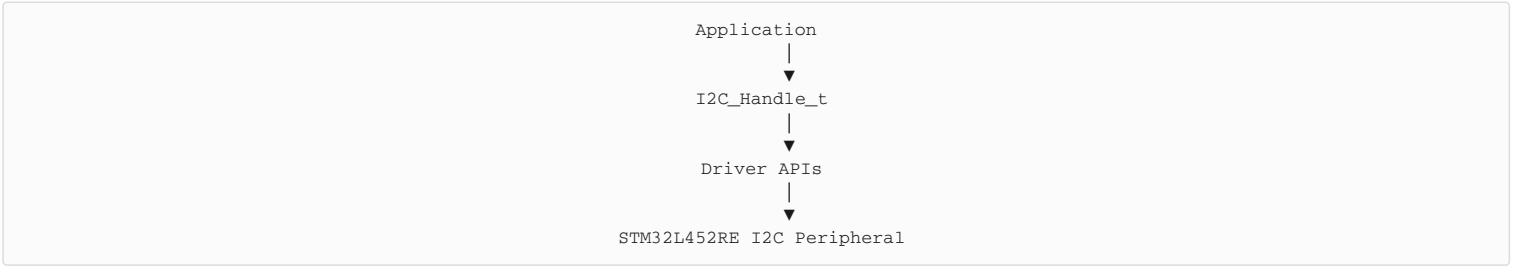
These structures simplify driver development by allowing a single handle to carry all the information required by the driver.

1. Why Are These Structures Required?

A driver must know:

- Which I2C peripheral is being used (I2C1, I2C2, or I2C3).
- How the peripheral should be configured.
- The current state of the peripheral.
- Information needed during interrupt-driven communication.

Instead of passing many parameters to every API, all information is stored in one handle.



2. I2C Configuration Structure (I2C_Config_t)

The configuration structure stores every option that the application can configure before calling `I2C_Init()`.

```
typedef struct
{
    uint32_t I2C_Timing;

    uint16_t I2C_OwnAddress;

    uint8_t I2C_AddressingMode;

    uint8_t I2C_ACKControl;

    uint8_t I2C_AnalogFilter;

    uint8_t I2C_DigitalFilter;

}I2C_Config_t;
```

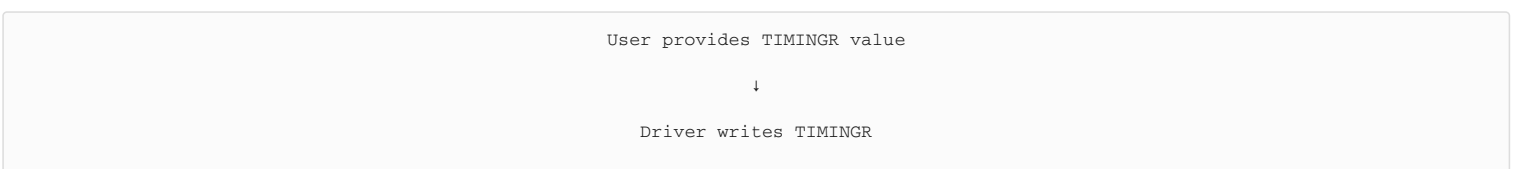
3. Description of Each Member

Member	Data Type	Purpose
I2C_Timing	uint32_t	TIMINGR register value
I2C_OwnAddress	uint16_t	MCU slave address
I2C_AddressingMode	uint8_t	7-bit or 10-bit addressing
I2C_ACKControl	uint8_t	Enable/Disable automatic ACK
I2C_AnalogFilter	uint8_t	Enable/Disable analog filter
I2C_DigitalFilter	uint8_t	Number of digital filter clock cycles

4. Understanding Each Configuration Parameter

A. I2C_Timing

This is the biggest architectural change in the timing configuration approach:



```
                    ↓

        Hardware generates SCL
```

The timing value controls: PRESC, SCLDEL, SDADEL, SCLH, SCLL. These are explained in Lecture 6.

Example

```
I2CHandle.I2C_Config.I2C_Timing = 0x10909CEC;
```

This value typically corresponds to 100 kHz operation for a specific peripheral clock configuration. It depends on the I2C kernel clock frequency and should be calculated or generated using ST's timing configuration method.

B. Own Address

This address is only used when the STM32L452RE operates as a slave.

Example

```
I2CHandle.I2C_Config.I2C_OwnAddress = 0x52;
```

The address is stored in the OAR1 register.

C. Addressing Mode

STM32L452RE supports 7-bit addressing and 10-bit addressing. The L4 hardware supports both modes directly.

7-bit Addressing – Most sensors, EEPROM, RTC, OLED, ADC, DAC

10-bit Addressing – Used when more slave devices are required, industrial applications, large embedded systems

5. ACK Control

Every received byte should normally be acknowledged. Possible options: ACK Enabled, ACK Disabled.

When ACK is enabled

```
                Master
                  ↓
            Receives Byte
                  ↓
        Automatically Sends ACK
                  ↓
        Slave Continues Transmission
```

When disabled

```
                Receive Byte
                     ↓
                NACK Generated
                     ↓
                Transfer Ends
```

6. Analog Filter

The STM32L452RE includes an internal analog filter.

Purpose: Removes very short glitches from SDA and SCL.

```
                    Noise
                      ↓
                Analog Filter
                      ↓
                Clean Signal
                      ↓
                I2C Peripheral
```

Benefits: Better EMI immunity, better noise rejection, more reliable communication. Normally enabled.

7. Digital Filter

The STM32L452RE also contains a programmable digital filter.

Purpose: Ignore very short spikes on SDA/SCL. The user selects 0, 1, 2 … 15 clock cycles.

Example: Digital Filter = 4 → Ignore glitches shorter than 4 clock cycles.

8. Handle Structure

The handle combines: peripheral pointer, configuration, driver state.

```
typedef struct
{
    I2C_RegDef_t *pI2Cx;
```

```
    I2C_Config_t I2C_Config;

}I2C_Handle_t;
```

This is the minimum structure. As we implement interrupt-driven communication later, we'll extend it with additional members such as transmit/receive buffers and state variables.

9. Understanding the Handle

```
                    I2C_Handle_t
+---------------------------------+
                    pI2Cx
                      ↓
                    I2C1
                    I2C2
                    I2C3
                      +
                  I2C_Config
                      ↓
                   Timing
                 Own Address
                   Filters
                     ACK
                 Address Mode
+---------------------------------+
```

Every API receives only one parameter:

```
I2C_Init(&I2CHandle);
```

instead of:

```
I2C_Init(
    I2C1,
    100kHz,
    ACK,
    OwnAddress,
    Filter,
    AddressMode
);
```

which would become difficult to maintain.

10. Typical Application Code

```
I2C_Handle_t I2CHandle;

I2CHandle.pI2Cx = I2C1;

I2CHandle.I2C_Config.I2C_Timing = 0x10909CEC;

I2CHandle.I2C_Config.I2C_OwnAddress = 0x52;

I2CHandle.I2C_Config.I2C_AddressingMode = I2C_ADDRMODE_7BIT;

I2CHandle.I2C_Config.I2C_ACKControl = I2C_ACK_ENABLE;

I2CHandle.I2C_Config.I2C_AnalogFilter = ENABLE;

I2CHandle.I2C_Config.I2C_DigitalFilter = 0;

I2C_Init(&I2CHandle);
```

11. Future Expansion of the Handle

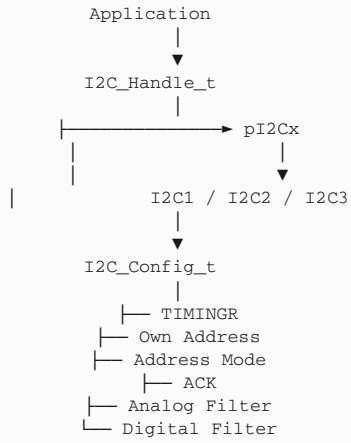
When we implement interrupt-driven communication, the handle will become:

```
typedef struct
{
    I2C_RegDef_t *pI2Cx;

    I2C_Config_t I2C_Config;

    uint8_t *pTxBuffer;

    uint8_t *pRxBuffer;

    uint32_t TxLen;

    uint32_t RxLen;

    uint8_t TxState;
```

```
    uint8_t RxState;

    uint8_t DevAddr;

    uint32_t RxSize;

    uint8_t Sr;

}I2C_Handle_t;
```

This mirrors how professional STM32 drivers maintain communication state without using global variables.

12. Relationship Between Structures



Key Takeaways

- The I2C_Config_t structure stores all user-configurable settings, including TIMINGR, own address, addressing mode, ACK control, and analog/digital filter settings.
- The I2C_Handle_t structure combines the selected I2C peripheral pointer with its configuration, allowing driver APIs to work with a single handle.
- The STM32L452RE uses the TIMINGR register instead of separate CCR and TRISE registers for I2C timing configuration.
- The handle structure is designed to be extended later with transmit/receive buffers, transfer lengths, and state variables for interrupt-driven communication.

In Lecture 3, we'll define all the user configuration macros (TIMINGR presets, ACK, addressing mode, filter options, and common configuration constants) specifically for the STM32L452RE.

Lecture 3 — Configuration Macros for User Options

Lecture Objective

In this lecture, we will:

- Include the required header files.
- Connect the driver with the MCU-specific header.
- Create user-friendly configuration macros.
- Define macros for timing, addressing mode, ACK control, analog filter, digital filter, and transfer modes.
- Understand why these macros improve readability and maintainability.

The STM32L452RE requires macros for TIMINGR values and additional hardware features.

1. Include Required Header Files

In `stm32l1452xx_i2c_driver.h`:

```
#include "stm32l452xx.h"
```

Purpose: This provides access to:

- Register definitions
- Peripheral base addresses
- RCC macros
- Standard data types
- Bit position macros

2. Include Driver Header

At the end of `stm32l452xx.h`:

```
#include "stm32l452xx_i2c_driver.h"
```

This allows every source file including `stm32l1452xx.h` to automatically access the I2C driver definitions.

3. Driver Source File

At this stage, `stm32l1452xx_i2c_driver.c` contains only:

```
#include "stm32l452xx_i2c_driver.h"
```

API implementations will be added in later lectures.

4. Why Use Configuration Macros?

Instead of writing numeric values everywhere:

✖ Bad

```
I2CHandle.I2C_Config.I2C_ACKControl = 1;
```

✔ Better

```
I2CHandle.I2C_Config.I2C_ACKControl = I2C_ACK_ENABLE;
```

Advantages: Improves readability, avoids magic numbers, simplifies debugging, makes code portable, easier to maintain.

5. Timing Configuration Macros

Timing is configured by programming the TIMINGR register. Typical presets are:

```
/*=====================================================
 * I2C Timing Values
 * (Examples only - depend on I2C kernel clock)
 *=====================================================*/

#define I2C_TIMING_100KHZ     0x10909CEC
#define I2C_TIMING_400KHZ     0x00702991
#define I2C_TIMING_1MHZ       0x00300B29
```

> **Important:** These values are examples. The correct TIMINGR value depends on the I2C kernel clock (e.g., 16 MHz, 48 MHz, 80 MHz). They should be calculated according to RM0394 or generated using STM32CubeMX or ST's timing configuration tool.

Timing Summary

Macro	Typical Speed
I2C_TIMING_100KHZ	Standard Mode
I2C_TIMING_400KHZ	Fast Mode
I2C_TIMING_1MHZ	Fast Mode Plus

6. Addressing Mode Macros

STM32L452RE supports both addressing modes.

```
#define I2C_ADDRMODE_7BIT      0
#define I2C_ADDRMODE_10BIT     1
```

7-bit Addressing – Most commonly used for: EEPROM, RTC, OLED, temperature sensors, accelerometers.

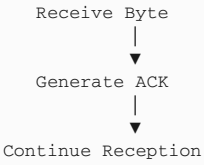
10-bit Addressing – Used when: large systems, industrial applications, many slave devices.

7. ACK Control

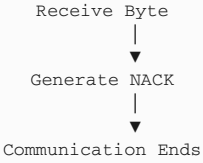
The ACK bit controls whether the peripheral automatically acknowledges received bytes.

```
#define I2C_ACK_ENABLE         1
#define I2C_ACK_DISABLE        0
```

When ACK is enabled:



When ACK is disabled:



8. Analog Filter Macros

The analog filter removes short noise pulses on SDA and SCL.

```
#define I2C_ANALOG_FILTER_ENABLE     1
#define I2C_ANALOG_FILTER_DISABLE    0
```

Recommended: Enable Analog Filter unless very high-speed or timing-critical applications require otherwise.

9. Digital Filter Macros

The STM32L452RE digital filter can suppress glitches lasting up to 15 I2C kernel clock cycles.

```
#define I2C_DIGITAL_FILTER_0      0
#define I2C_DIGITAL_FILTER_1      1
#define I2C_DIGITAL_FILTER_2      2
...
#define I2C_DIGITAL_FILTER_15     15
```

Example

```
I2CHandle.I2C_Config.I2C_DigitalFilter =
I2C_DIGITAL_FILTER_4;
```

This ignores glitches shorter than four kernel clock cycles.

10. Transfer Mode Macros

STM32L452RE supports different transfer modes through the CR2 register.

```
#define I2C_SOFTEND_MODE      0
#define I2C_AUTOEND_MODE      1
```

SoftEnd Mode

```
Transfer Complete
        ↓
Software Generates STOP
```

Useful for repeated START conditions.

AutoEnd Mode

```
Transfer Complete
        ↓
Hardware Automatically Generates STOP
```

Useful for simple master transactions.

11. Reload Mode Macros

The hardware also supports automatic reload.

```
#define I2C_RELOAD_DISABLE      0
#define I2C_RELOAD_ENABLE       1
```

Used for transfers larger than 255 bytes, allowing the software to update the transfer count during communication.

12. Complete Configuration Macro List

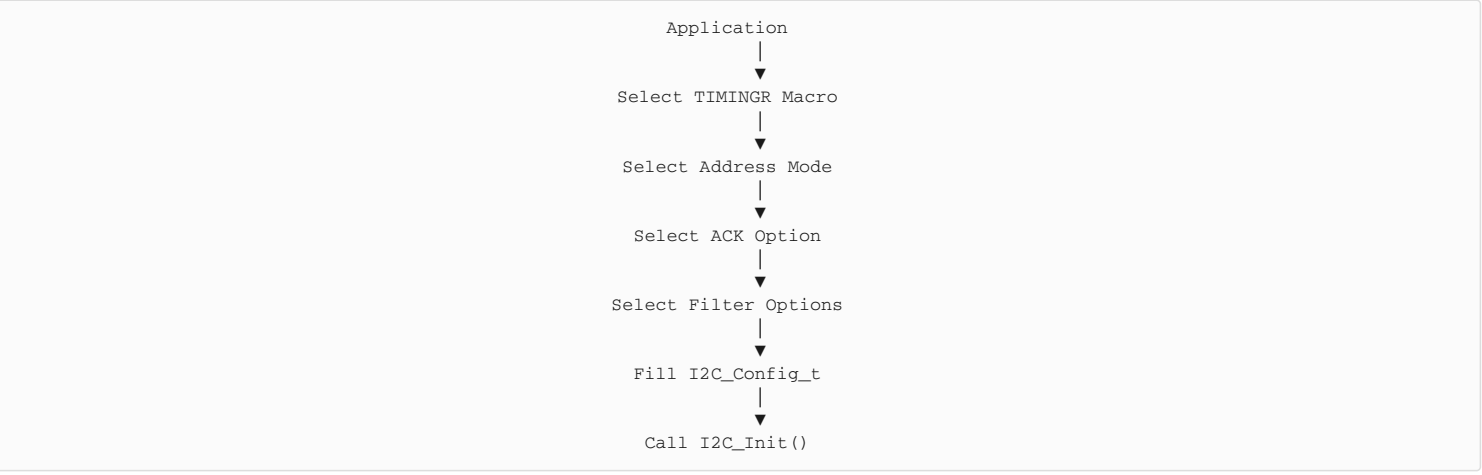
```
/*=====================================================
 * TIMINGR Configuration
 *=====================================================*/

#define I2C_TIMING_100KHZ      0x10909CEC
#define I2C_TIMING_400KHZ      0x00702991
#define I2C_TIMING_1MHZ        0x00300B29

/*=====================================================
 * Addressing Mode
 *=====================================================*/

#define I2C_ADDRMODE_7BIT      0
#define I2C_ADDRMODE_10BIT     1

/*=====================================================
 * ACK
 *=====================================================*/

#define I2C_ACK_ENABLE         1
#define I2C_ACK_DISABLE        0

/*=====================================================
 * Analog Filter
 *=====================================================*/

#define I2C_ANALOG_FILTER_ENABLE      1
#define I2C_ANALOG_FILTER_DISABLE     0

/*=====================================================
 * Digital Filter
 *=====================================================*/

#define I2C_DIGITAL_FILTER_0          0
#define I2C_DIGITAL_FILTER_15         15

/*=====================================================
 * Transfer Modes
 *=====================================================*/

#define I2C_SOFTEND_MODE              0
#define I2C_AUTOEND_MODE              1

/*=====================================================
 * Reload
 *=====================================================*/

#define I2C_RELOAD_DISABLE            0
#define I2C_RELOAD_ENABLE             1
```

13. Example Usage

```
I2C_Handle_t I2CHandle;

I2CHandle.pI2Cx = I2C1;

I2CHandle.I2C_Config.I2C_Timing = I2C_TIMING_100KHZ;
```

```
I2CHandle.I2C_Config.I2C_OwnAddress = 0x52;

I2CHandle.I2C_Config.I2C_AddressingMode = I2C_ADDRMODE_7BIT;

I2CHandle.I2C_Config.I2C_ACKControl = I2C_ACK_ENABLE;

I2CHandle.I2C_Config.I2C_AnalogFilter = I2C_ANALOG_FILTER_ENABLE;

I2CHandle.I2C_Config.I2C_DigitalFilter = I2C_DIGITAL_FILTER_0;
```

Configuration Flow



Key Takeaways

- The STM32L452RE uses TIMINGR presets instead of CCR and TRISE for I2C timing configuration.
- Configuration macros improve readability, reduce errors, and simplify driver maintenance.
- New-generation I2C peripherals support additional features such as analog filtering, digital filtering, AUTOEND, and RELOAD, which should be exposed through user-friendly macros.
- Timing values are dependent on the I2C kernel clock and must be chosen or calculated appropriately for the target clock frequency.

Note for this series: *For a production-quality driver, we'll avoid hardcoded TIMINGR values where possible. Later in the series, we'll discuss how to derive or validate them from the STM32L452RE reference manual and clock configuration, so the driver remains portable across different system clock settings.*

Lecture 4 — Creating API Prototypes

Lecture Objective

In this lecture, we will:

- Design the public APIs for the STM32L452RE I2C driver.
- Reuse the common driver architecture developed for GPIO and SPI.
- Add APIs specific to the STM32L4 I2C peripheral.
- Prepare the driver for both polling and interrupt-based communication.

By the end of this lecture, the driver interface will be complete, while the implementation will be developed in subsequent lectures.

1. Why API Prototypes?

A driver acts as the interface between the application and the hardware. Instead of directly manipulating registers like this:

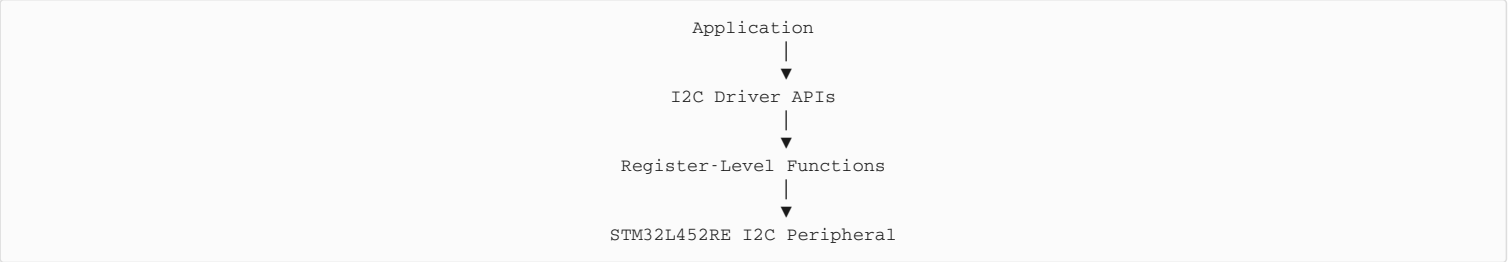
```
I2C1->CR1 |= (1 << 0);
```

the application should simply call:

```
I2C_PeripheralControl(I2C1, ENABLE);
```

Advantages: Easier to read, hardware-independent application code, better portability, simplified debugging, modular software design.

2. Driver Architecture



The application should never access hardware registers directly.

3. Categories of APIs

The driver APIs can be grouped into six categories:

Category	Purpose
Clock Control	Enable/Disable peripheral clock
Initialization	Configure peripheral
Communication	Master/Slave data transfer
Peripheral Control	Enable/Disable I2C
Interrupt Control	Configure NVIC and interrupts
Status APIs	Read peripheral flags

4. Peripheral Clock Control

Prototype

```
void I2C_PeriClockControl(I2C_RegDef_t *pI2Cx,
                          uint8_t EnOrDi);
```

Purpose: Enable or disable the RCC clock for the selected peripheral.

Parameter	Description
pI2Cx	I2C1 / I2C2 / I2C3
EnOrDi	ENABLE or DISABLE

Typical use

```
I2C_PeriClockControl(I2C1, ENABLE);
```

5. Initialization API

```
void I2C_Init(I2C_Handle_t *pI2CHandle);
```

Purpose: Configures TIMINGR, Own Address, Address Mode, Analog Filter, Digital Filter, ACK, and Peripheral Enable. No CCR or TRISE calculations are performed.

6. De-Initialization

```
void I2C_DeInit(I2C_RegDef_t *pI2Cx);
```

Purpose: Returns the peripheral to its reset state.

Implementation: Assert peripheral reset, release peripheral reset – exactly like GPIO and SPI drivers.

7. Peripheral Enable API

```
void I2C_PeripheralControl(I2C_RegDef_t *pI2Cx,
                           uint8_t EnOrDi);
```

Purpose: Enable or disable the peripheral after configuration.

Hardware register: CR1, Bit 0, PE.

When PE = 0 → Peripheral Disabled. When PE = 1 → Peripheral Enabled.

8. Master Transmission API

Prototype

```
void I2C_MasterSendData(
        I2C_Handle_t *pHandle,
        uint8_t *pTxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);
```

Responsibilities: Configure CR2, generate START, send address, send bytes, wait for TXIS, monitor TC/TCR, generate STOP (or AUTOEND).

9. Master Reception API

```
void I2C_MasterReceiveData(
        I2C_Handle_t *pHandle,
        uint8_t *pRxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);
```

Responsibilities: Generate START, send slave address, read RXDR, wait for RXNE, generate STOP.

10. Slave Transmission API

```
void I2C_SlaveSendData(
        I2C_Handle_t *pHandle,
        uint8_t Data);
```

Unlike the master, the slave never generates START and never generates STOP. It simply waits until addressed by a master and then writes data to TXDR.

11. Slave Reception API

```
uint8_t I2C_SlaveReceiveData(
        I2C_Handle_t *pHandle);
```

Reads a received byte from RXDR after the master initiates communication.

12. Interrupt Configuration APIs

Exactly the same concept used in GPIO and SPI.

Interrupt Enable

```
void I2C_IRQInterruptConfig(
        uint8_t IRQNumber,
        uint8_t EnOrDi);
```

Purpose: Enable/Disable interrupt in NVIC.

Interrupt Priority

```
void I2C_IRQPriorityConfig(
        uint8_t IRQNumber,
        uint8_t Priority);
```

Purpose: Assign NVIC priority.

13. IRQ Handler

Prototype

```
void I2C_EV_IRQHandler(
        I2C_Handle_t *pHandle);

void I2C_ER_IRQHandler(
        I2C_Handle_t *pHandle);
```

STM32L452RE separates Event Interrupts and Error Interrupts. The handler will later process: TXIS, RXNE, STOPF, NACKF, TC, TCR, BERR, ARLO, OVR, TIMEOUT.

14. Flag Status API

```
uint8_t I2C_GetFlagStatus(
        I2C_RegDef_t *pI2Cx,
        uint32_t FlagName);
```

Purpose: Read status flags from the ISR register.

Common flags

Flag	Description
TXIS	TXDR empty
RXNE	RXDR contains data
BUSY	Bus busy
STOPF	STOP detected
NACKF	NACK received
TC	Transfer Complete
TCR	Transfer Reload
BERR	Bus Error
ARLO	Arbitration Lost

15. Interrupt-Based Communication APIs

Polling wastes CPU time. Professional drivers therefore provide:

```
uint8_t I2C_MasterSendDataIT(
        I2C_Handle_t *pHandle,
        uint8_t *pTxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);

uint8_t I2C_MasterReceiveDataIT(
        I2C_Handle_t *pHandle,
        uint8_t *pRxBuffer,
        uint32_t Len,
        uint16_t SlaveAddr);
```

These APIs configure interrupts, return immediately, and let the CPU continue other work. Communication completes through interrupts.

16. Close APIs

```
void I2C_CloseSendData(
        I2C_Handle_t *pHandle);

void I2C_CloseReceiveData(
        I2C_Handle_t *pHandle);
```

Purpose: Reset driver state, disable interrupts, prepare next transfer.

17. Application Callback

```
void I2C_ApplicationEventCallback(
        I2C_Handle_t *pHandle,
```

```
        uint8_t AppEvent);
```

Purpose: Notify the application when events occur.

Example events: I2C_EVENT_TX_COMPLETE, I2C_EVENT_RX_COMPLETE, I2C_EVENT_STOP, I2C_EVENT_NACK, I2C_EVENT_ERROR

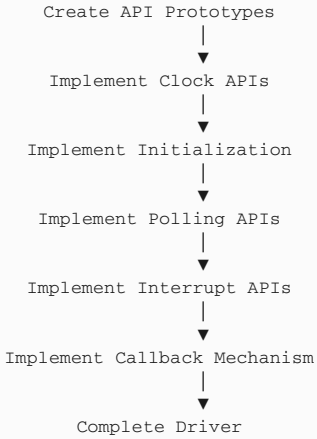
Example

```
void I2C_ApplicationEventCallback(
        I2C_Handle_t *pHandle,
        uint8_t Event)
{
    switch(Event)
    {
        case I2C_EVENT_TX_COMPLETE:
            break;

        case I2C_EVENT_RX_COMPLETE:
            break;
    }
}
```

18. Complete Driver API List

```
/* Clock */
I2C_PeriClockControl()

/* Initialization */
I2C_Init()
I2C_DeInit()

/* Peripheral */
I2C_PeripheralControl()

/* Polling Communication */
I2C_MasterSendData()
I2C_MasterReceiveData()
I2C_SlaveSendData()
I2C_SlaveReceiveData()

/* Interrupt Communication */
I2C_MasterSendDataIT()
I2C_MasterReceiveDataIT()

/* Interrupt */
I2C_IRQInterruptConfig()
I2C_IRQPriorityConfig()
I2C_EV_IRQHandler()
I2C_ER_IRQHandler()

/* Utility */
I2C_GetFlagStatus()
I2C_CloseSendData()
I2C_CloseReceiveData()
I2C_ApplicationEventCallback()
```

Driver Development Flow



Key Takeaways

- The STM32L452RE I2C driver follows the same modular architecture used for GPIO and SPI drivers, with APIs grouped into clock control, initialization, communication, interrupt handling, and utility functions.
- Communication APIs are divided into polling and interrupt-driven versions, allowing applications to choose between simplicity and

CPU efficiency.

- The driver uses the STM32L4-specific ISR, ICR, TXDR, and RXDR registers instead of older status and data registers.
- Event callback functions provide a clean mechanism for notifying the application when transfers complete or errors occur, making the driver suitable for larger embedded applications.

In Lecture 5, we'll begin implementing I2C_Init() for the STM32L452RE, covering the correct initialization sequence, RCC clock enabling, CR1 configuration, filter setup, own-address programming, and TIMINGR configuration before enabling the peripheral.

Lecture 5 — Implementing I2C_Init()

Lecture Objective

In this lecture, we will implement the `I2C_Init()` function for the STM32L452RE. The initialization sequence differs significantly from older families because the STM32L4 family uses:

- TIMINGR instead of CCR/TRISE
- CR1 for filter configuration
- OAR1 for own address
- ISR/ICR instead of SR1/SR2

By the end of this lecture, you'll understand how to correctly initialize the STM32L452RE I2C peripheral before communication begins.

1. Why Initialization Is Required

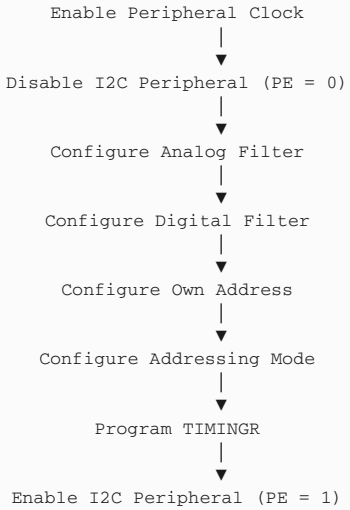
When the microcontroller powers up:

- The I2C peripheral clock is disabled.
- All I2C registers are in their reset state.
- The peripheral is disabled (PE = 0).
- Timing parameters are not configured.
- Own address is not programmed.

Before using I2C, all required registers must be configured.

2. Initialization Sequence

The recommended initialization sequence for the STM32L452RE is:



> **Important:** The PE bit in CR1 must be cleared before modifying configuration registers such as TIMINGR, OAR1, or filter settings. This requirement is specified in the STM32L4 reference manual.

3. Function Prototype

```
void I2C_Init(I2C_Handle_t *pI2CHandle);
```

Parameter: pI2CHandle – Pointer to the I2C handle containing the peripheral instance and configuration parameters.

4. Step 1 – Enable Peripheral Clock

Before accessing the peripheral registers, enable its clock.

```
if(pI2CHandle->pI2Cx == I2C1)
{
    I2C1_PCLK_EN();
}
else if(pI2CHandle->pI2Cx == I2C2)
{
    I2C2_PCLK_EN();
}
else if(pI2CHandle->pI2Cx == I2C3)
{
    I2C3_PCLK_EN();
}
```

Without enabling the peripheral clock: register writes have no effect, and the peripheral cannot operate.

5. Step 2 – Disable the Peripheral

The STM32L452RE requires the peripheral to be disabled before changing configuration registers.

```
pI2CHandle->pI2Cx->CR1 &= ~(1U << I2C_CR1_PE_Pos);
```

Why? Some configuration registers are write-protected while PE = 1. These include: TIMINGR, OAR1, OAR2, CR1 filter bits.

6. Step 3 – Configure the Analog Filter

The analog filter removes short glitches on the SDA and SCL lines.

If enabled:

```
pI2CHandle->pI2Cx->CR1 &= ~(1U << I2C_CR1_ANFOFF_Pos);
```

If disabled:

```
pI2CHandle->pI2Cx->CR1 |= (1U << I2C_CR1_ANFOFF_Pos);
```

Recommendation: Leave the analog filter enabled unless your application has specific timing requirements.

7. Step 4 – Configure the Digital Filter

The digital filter suppresses glitches lasting fewer than a programmable number of I2C kernel clock cycles. The DNF field occupies bits 11:8 of the CR1 register.

```
pI2CHandle->pI2Cx->CR1 &= ~(0xFU << I2C_CR1_DNF_Pos);

pI2CHandle->pI2Cx->CR1 |=
(
    pI2CHandle->I2C_Config.I2C_DigitalFilter
    << I2C_CR1_DNF_Pos
);
```

Valid values:

Value	Meaning
0	Digital filter disabled
1–15	Filter length in kernel clock cycles

8. Step 5 – Configure Own Address

The own address is required only when the STM32 acts as an I2C slave. The address is programmed into OAR1.

```
uint32_t temp = 0;

temp |= (1U << 15);     // OA1EN

temp |=
(
    pI2CHandle->I2C_Config.I2C_OwnAddress << 1
);

pI2CHandle->pI2Cx->OAR1 = temp;
```

The OA1EN bit enables Own Address 1.

9. Step 6 – Configure Addressing Mode

STM32L452RE supports:

Mode	Description
7-bit	Most common
10-bit	Larger address space

For 10-bit addressing:

```
pI2CHandle->pI2Cx->OAR1 |=
(1U << I2C_OAR1_OA1MODE_Pos);
```

For 7-bit addressing:

```
pI2CHandle->pI2Cx->OAR1 &=
~(1U << I2C_OAR1_OA1MODE_Pos);
```

10. Step 7 – Program TIMINGR

There is no CCR register. Instead:

```
pI2CHandle->pI2Cx->TIMINGR =
pI2CHandle->I2C_Config.I2C_Timing;
```

This single register controls: PRESC, SCLDEL, SDADEL, SCLH, SCLL. We'll study how these fields are calculated in the next lecture.

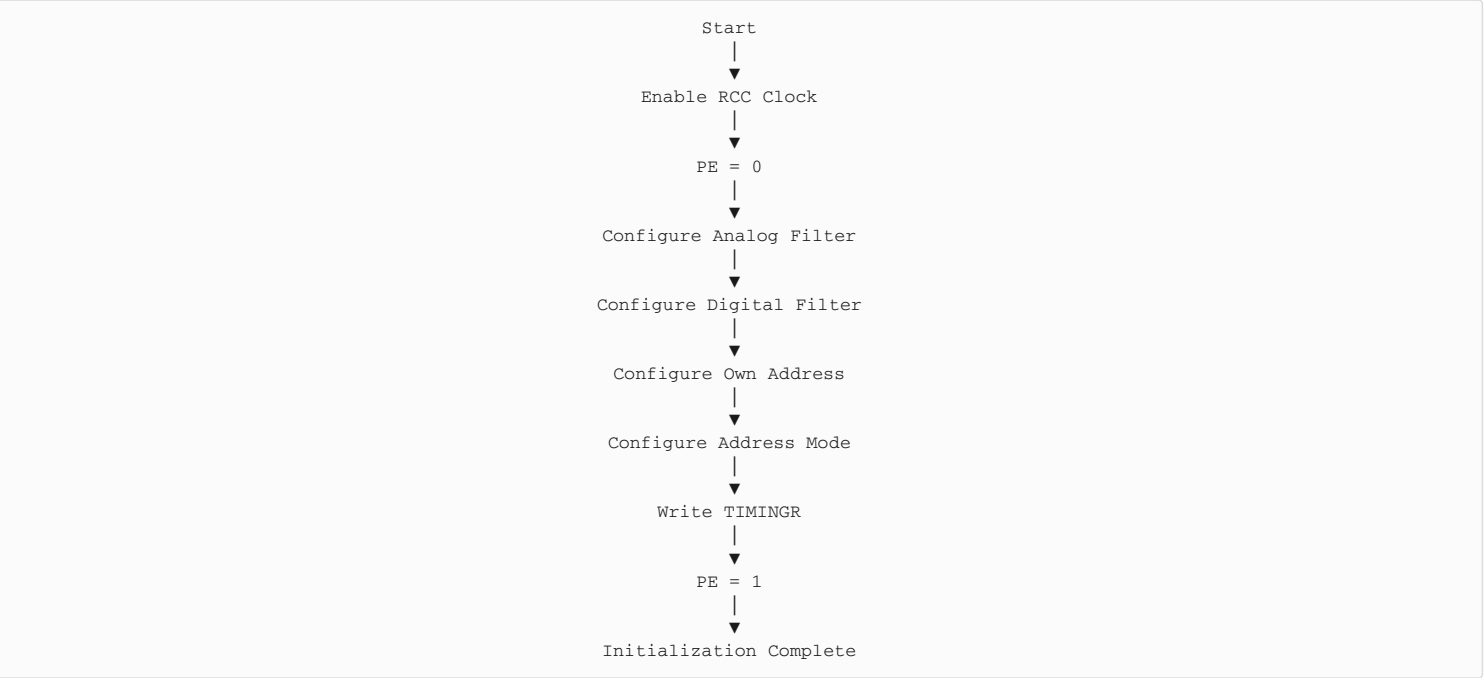
11. Step 8 – Enable the Peripheral

Finally, enable the I2C peripheral.

```
pI2CHandle->pI2Cx->CR1 |=
(1U << I2C_CR1_PE_Pos);
```

Now the peripheral is ready for communication.

12. Complete I2C_Init() Flow



13. Pseudocode

```
void I2C_Init(...)
{
    Enable Clock;

    Disable Peripheral;

    Configure Analog Filter;

    Configure Digital Filter;

    Configure Own Address;

    Configure Address Mode;

    Program TIMINGR;

    Enable Peripheral;
}
```

14. Common Mistakes

Forgetting to Enable the RCC Clock – The peripheral remains inaccessible.

Programming TIMINGR While PE = 1 – The write may be ignored or lead to incorrect behavior.

Using an Incorrect TIMINGR Value – Symptoms: no ACK from slave, bus remains BUSY, communication errors, incorrect SCL frequency. Always ensure the TIMINGR value matches the I2C kernel clock frequency.

Forgetting to Enable OA1EN – The STM32 will not respond to its own slave address.

Complete I2C_Init() Skeleton

```
void I2C_Init(I2C_Handle_t *pI2CHandle)
{
    /* 1. Enable peripheral clock */

    /* 2. Disable peripheral (PE = 0) */

    /* 3. Configure analog filter */

    /* 4. Configure digital filter */

    /* 5. Configure own address */

    /* 6. Configure addressing mode */

    /* 7. Program TIMINGR */

    /* 8. Enable peripheral */
}
```

Key Takeaways

- The STM32L452RE initialization sequence differs substantially from older families due to the redesigned I2C peripheral.
- Configuration registers such as TIMINGR, OAR1, and filter settings must be programmed while the peripheral is disabled (PE = 0).
- The TIMINGR register replaces the older CCR and TRISE registers, consolidating all timing configuration into a single register.
- Proper configuration of the analog filter, digital filter, and own address ensures reliable communication and correct slave operation.

The next lecture is a dedicated chapter explaining the TIMINGR register (PRESC, SCLDEL, SDADEL, SCLH, and SCLL), which is the correct way to configure I2C timing on the STM32L4 family.

Lecture 6 — I2C Timing Configuration Using TIMINGR

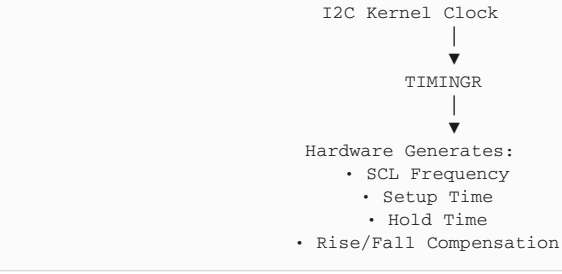
Lecture Objective

In this lecture, we will learn:

- Why the STM32L452RE uses the TIMINGR register.
- The purpose of each TIMINGR field.
- How the I2C clock is generated.
- How to configure Standard Mode (100 kHz), Fast Mode (400 kHz), and Fast Mode Plus (1 MHz).
- Why ST recommends using timing tools for TIMINGR values.

All I2C timing parameters are controlled by a single 32-bit TIMINGR register.

1. Why TIMINGR?



Everything is controlled by one register.

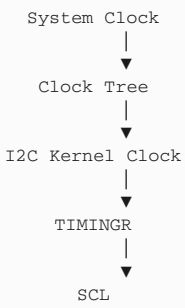
2. TIMINGR Register Layout

3. TIMINGR Fields

Field	Bits	Purpose
PRESC	31:28	Prescaler for I2C kernel clock
SCLDEL	23:20	SCL data setup delay
SDADEL	19:16	SDA data hold delay
SCLH	15:8	SCL HIGH period
SCLL	7:0	SCL LOW period

4. I2C Kernel Clock

The I2C peripheral is driven by an I2C kernel clock.



> **Note:** The kernel clock is selected through the STM32L4 clock tree and may differ from the APB1 clock, depending on the RCC configuration.

5. PRESC (Prescaler)

The PRESC field divides the I2C kernel clock.

Formula:

```
tPRESC = (PRESC + 1) × tI2CCLK
```

Example:

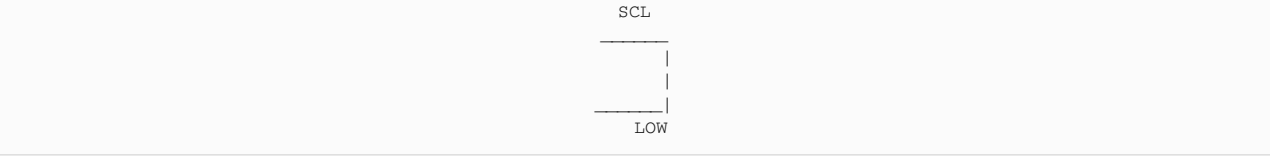
Kernel Clock = 80 MHz, Clock Period = 12.5 ns. If PRESC = 3, then:

```
tPRESC = (3 + 1) × 12.5 ns
       = 50 ns
```

All remaining timing calculations use this prescaled period.

6. SCLL (Low Period)

SCLL determines how long the SCL signal remains LOW.



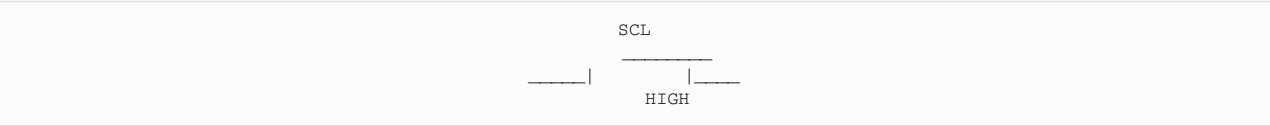
Approximate relationship:

```
LOW Time ≈ (SCLL + 1) × tPRESC
```

Increasing SCLL increases the LOW duration of the clock.

7. SCLH (High Period)

SCLH controls the HIGH portion of the SCL waveform.



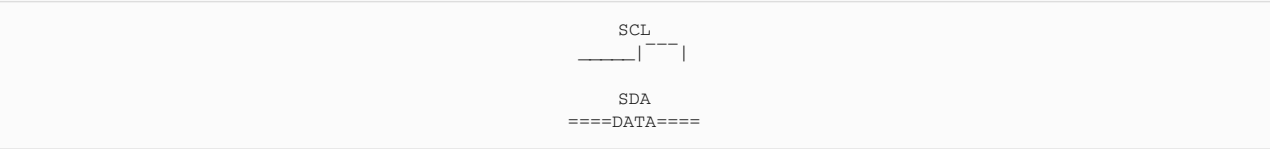
Approximate relationship:

```
HIGH Time ≈ (SCLH + 1) × tPRESC
```

Increasing SCLH increases the HIGH duration.

8. SDADEL (Data Hold Time)

After SCL changes, SDA must remain stable for a minimum time.



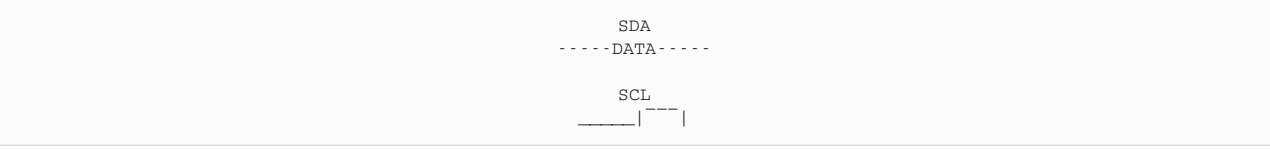
SDADEL ensures the required data hold time is met.

Approximate relationship:

```
Hold Time ≈ SDADEL × tPRESC
```

9. SCLDEL (Data Setup Time)

Before SCL changes, SDA must already be stable.



Approximate relationship:

```
Setup Time ≈ (SCLDEL + 1) × tPRESC
```

10. Putting It All Together

11. Typical Timing Values

For a common STM32L452RE configuration, example TIMINGR values are:

I2C Speed	Example TIMINGR*
100 kHz	0x10909CEC
400 kHz	0x00702991
1 MHz	0x00300B29

**These values are valid only for specific kernel clock frequencies. They must be recalculated if the I2C kernel clock changes.*

12. Programming TIMINGR

During initialization:

```
pI2CHandle->pI2Cx->TIMINGR =
pI2CHandle->I2C_Config.I2C_Timing;
```

No other timing registers need to be programmed.

13. Choosing the Correct TIMINGR Value

The TIMINGR value depends on:

- I2C kernel clock frequency
- Desired bus speed
- Rise time of SDA/SCL
- Fall time of SDA/SCL
- Analog filter setting
- Digital filter setting
- I2C specification timing requirements

Because these parameters interact, ST recommends generating TIMINGR values using STM32CubeMX's I2C Timing Configuration Tool, then validating them against RM0394.

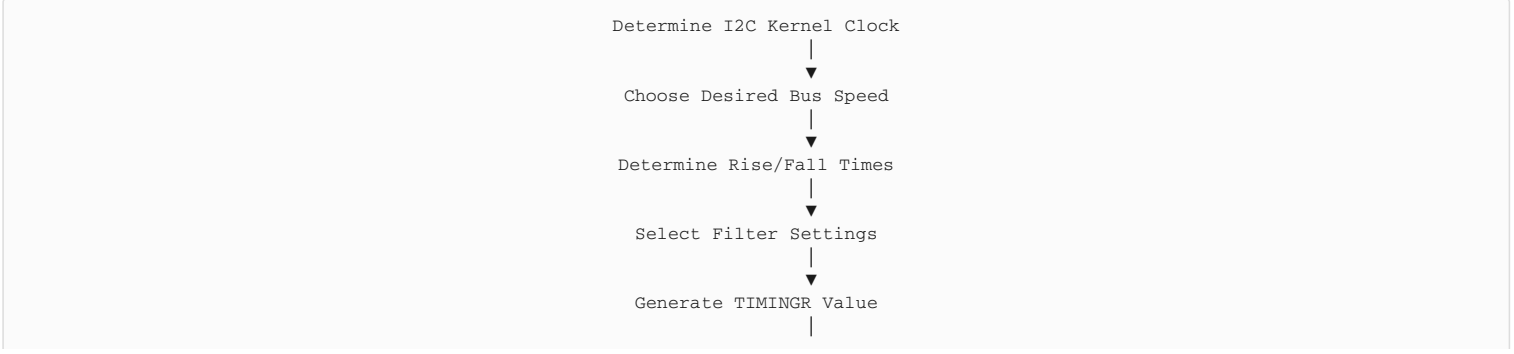
14. Common Mistakes

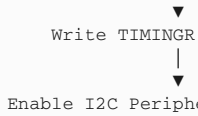
Using a TIMINGR value for the wrong kernel clock – A value generated for 80 MHz will not produce the correct bus speed if the kernel clock is 16 MHz.

Assuming TIMINGR is universal – It is not. Always match the TIMINGR value to the actual kernel clock.

Modifying TIMINGR while PE = 1 – The reference manual requires the peripheral to be disabled before changing timing configuration.

Timing Configuration Flow





Key Takeaways

- The TIMINGR register completely replaces the older CR2, CCR, and TRISE timing configuration.
- TIMINGR contains five fields: PRESC, SCLDEL, SDADEL, SCLH, and SCLL, which together define the I2C bus timing.
- TIMINGR values are dependent on the I2C kernel clock, desired bus speed, and physical bus characteristics. A value valid for one clock configuration cannot be assumed valid for another.
- In practice, TIMINGR values are typically generated using ST's timing configuration tools and then programmed directly into the TIMINGR register during initialization.

Lecture 7 — I2C Clock Stretching

Lecture Objective

In this lecture, we will learn:

- What clock stretching is.
- Why clock stretching is required in I2C communication.
- How the STM32L452RE hardware performs clock stretching.
- The role of the NOSTRETCH bit in the CR1 register.
- Practical situations where clock stretching occurs.

1. What is Clock Stretching?

Clock stretching is an important feature of the I2C protocol that allows a slower device to temporarily pause communication.

Definition: Clock stretching is the process of holding the SCL (Serial Clock) line LOW to delay the next clock pulse.

While SCL remains LOW:

- No new clock pulses are generated.
- No new data bits are transferred.
- Communication is paused.
- The bus resumes operation only after the device releases SCL.

2. Why is Clock Stretching Needed?

The I2C master usually determines the communication speed. However, the slave device may occasionally require additional time to:

- Process received data.
- Read internal EEPROM or Flash memory.
- Perform sensor measurements.
- Complete internal calculations.
- Prepare the next data byte.

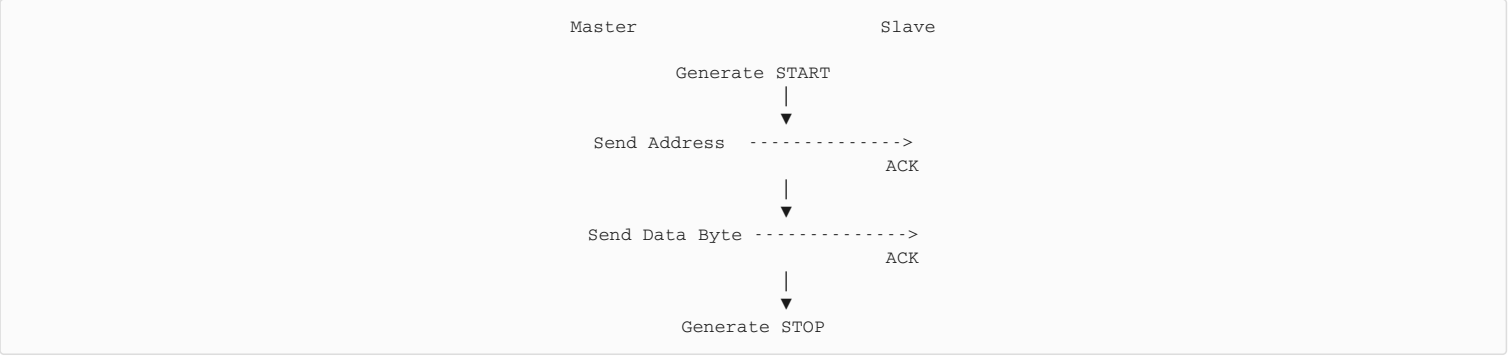
Instead of losing data, the slave requests extra time by stretching the clock.

3. Master and Slave Responsibilities

Device	Responsibility
Master	Generates the SCL clock and initiates transfers
Slave	May hold SCL LOW if it needs more processing time

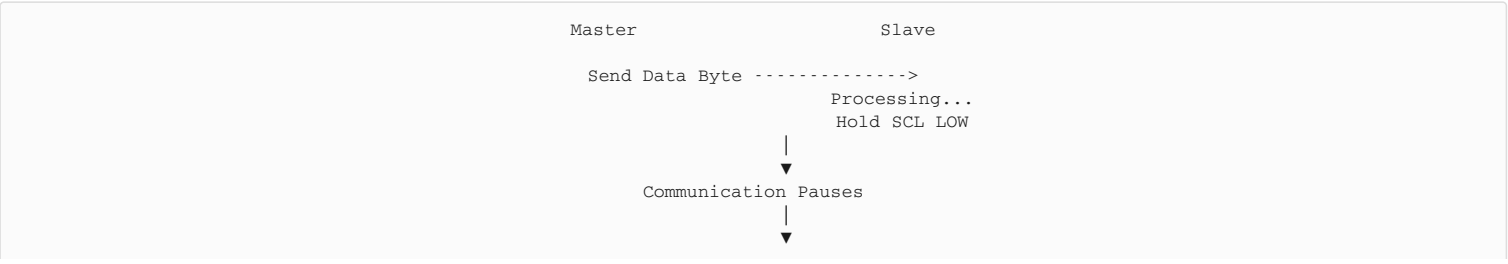
Even though the master generates the clock, the slave can delay the next rising edge by keeping the clock line LOW.

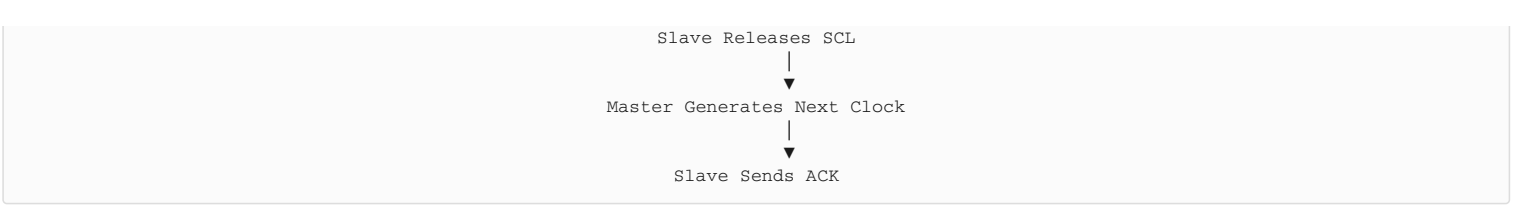
4. Normal Communication (No Clock Stretching)



Since the slave is ready, communication proceeds continuously.

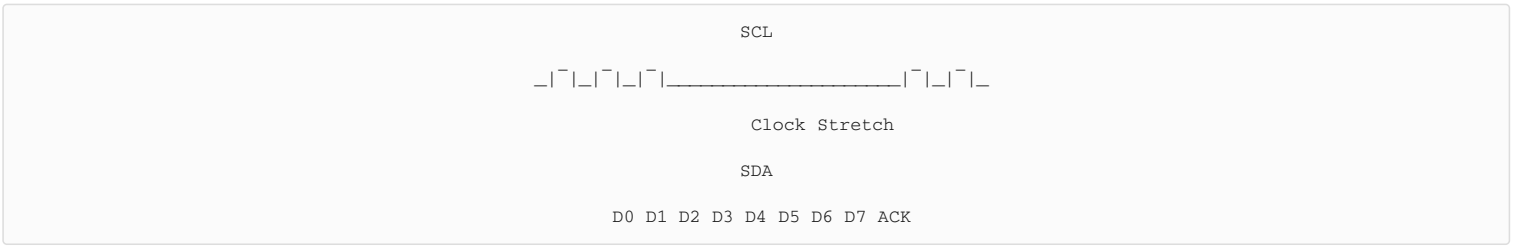
5. Communication with Clock Stretching





Communication resumes exactly where it stopped.

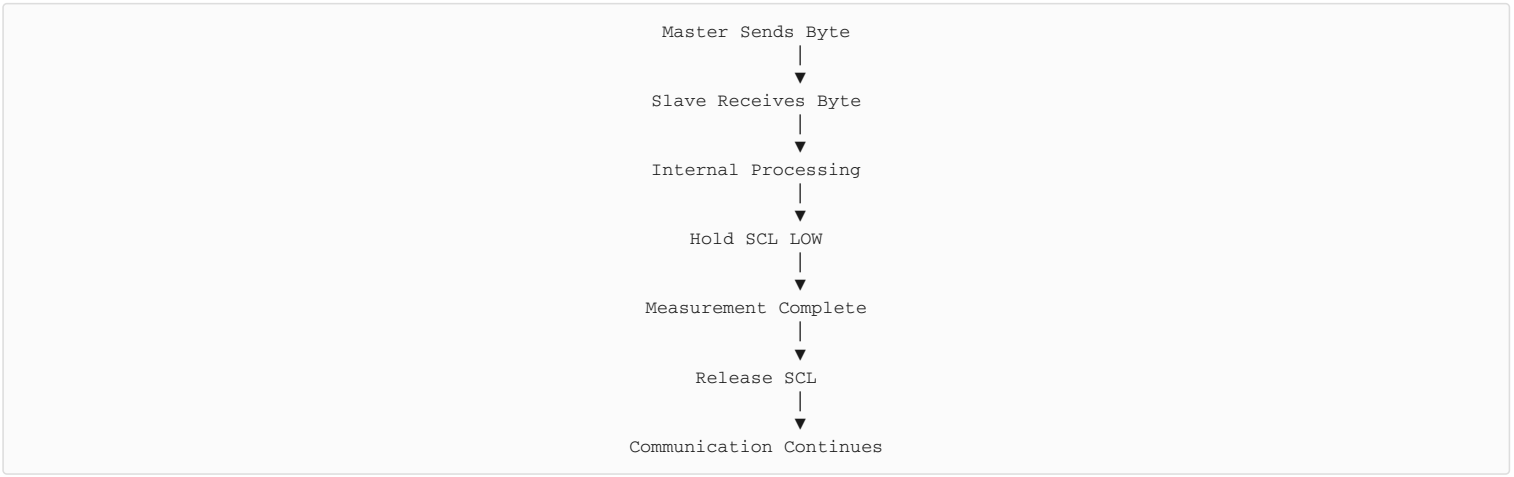
6. Timing Diagram



The long LOW period represents the slave stretching the clock.

7. What Happens Internally?

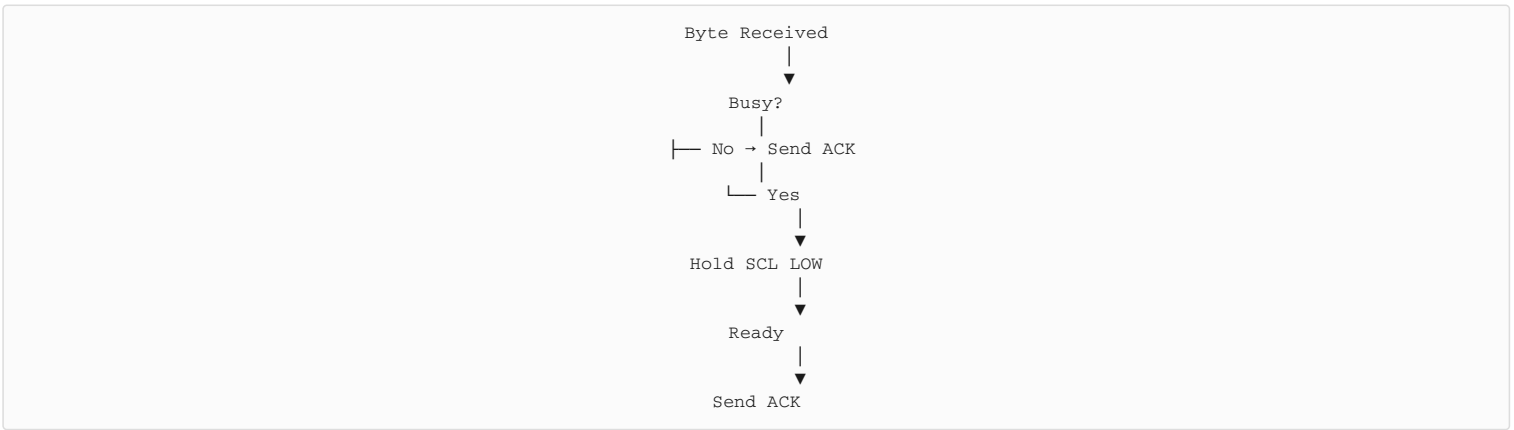
Suppose a temperature sensor needs time to update its measurement.



8. ACK and Clock Stretching

Recall that: 8 clock pulses transfer one data byte; the 9th clock pulse transfers the ACK/NACK bit.

If the slave is not ready to acknowledge immediately, it stretches the clock before the 9th pulse.



Without clock stretching, the master might interpret the delay as a NACK, causing the transfer to fail.

9. Clock Stretching in STM32L452RE

The STM32L452RE hardware automatically supports clock stretching. The software does not need to manually drive the SCL pin.

The hardware automatically:

- Holds SCL LOW when necessary.
- Waits until the peripheral is ready.
- Releases SCL.
- Continues communication.

10. The NOSTRETCH Bit

Clock stretching is controlled by the NOSTRETCH bit in the CR1 register.

NOSTRETCH	Meaning
0	Clock stretching enabled (default)
1	Clock stretching disabled

Example:

```
/* Enable clock stretching */
I2C1->CR1 &= ~(1U << I2C_CR1_NOSTRETCH_Pos);

/* Disable clock stretching */
I2C1->CR1 |= (1U << I2C_CR1_NOSTRETCH_Pos);
```

Recommendation: For most applications, leave clock stretching enabled. Disabling it is useful only in specific systems where all devices are guaranteed to respond within the required timing.

11. Why Not Disable Clock Stretching?

Disabling clock stretching can cause:

- Missed ACKs.
- Data corruption.
- Communication failures with slower slave devices.
- Incompatibility with many commercial I2C sensors and EEPROMs.

Most standard I2C devices expect the master to support clock stretching.

12. Devices That Commonly Use Clock Stretching

Many I2C peripherals use clock stretching internally, including:

- EEPROMs
- Temperature sensors
- Pressure sensors
- ADCs
- DACs
- Fuel gauge ICs
- Touch controllers
- Real-Time Clock (RTC) devices

These devices may need additional time before responding to the master.

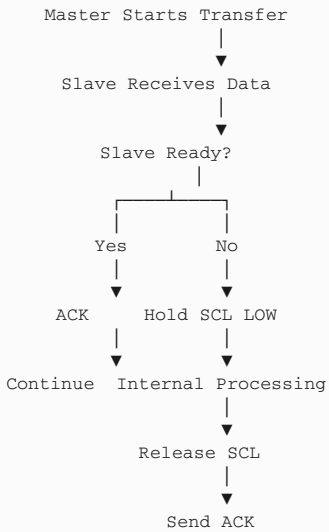
13. Common Misconceptions

"The master always controls the clock." Partially true. The master initiates the clock, but the slave can delay it by holding SCL LOW.

"Clock stretching is software-controlled." False. On the STM32L452RE, the hardware manages clock stretching automatically. Software only enables or disables the feature through the NOSTRETCH bit.

"Clock stretching slows down communication." Only when necessary. During normal transfers, no stretching occurs, and communication proceeds at the configured bus speed.

Communication Flow



Key Takeaways

- Clock stretching allows an I2C device—typically the slave—to pause communication by holding the SCL line LOW until it is ready to continue.
- The STM32L452RE hardware automatically performs clock stretching; software only controls whether the feature is enabled through the CR1.NOSTRETCH bit.
- Keeping clock stretching enabled ensures compatibility with many commercial I2C devices such as EEPROMs, sensors, ADCs, DACs, and RTCs.
- Clock stretching improves communication reliability by preventing missed acknowledgements and data loss when a slave requires additional processing time.